

Large Language Models - Exam

Politecnico di Torino

29 Gennaio 2025

Tempo impiegato: 1 ora 15 min. **Valutazione:** 15,50 / 18,00

Domanda 1 (1 point (-25% penalty for each wrong answer))

Which of the following are benefits of a Retrieval-Augmented-Generation (RAG) approach to prompting?

- a. It is possible to adopt self-critique mechanisms to refine the outputs of the agent.
- b. It is possible to reduce the amount of false information provided to the final user.
- c. It is possible to prevent the obsolescence of answers by using up-to-date documents.
- d. It is possible to reduce the amount of hallucinations by providing a fixed structure for the agent's answers.
- e. It is possible to decompose a task in several smaller steps.

Domanda 2 (1 point (-25% penalty for each wrong answer))

In-context Learning has been a fundamental paradigm shift for Language Models because:

- a. It replaces the need for attention mechanisms by introducing a sequential input processing.
- b. It improves the efficiency of training by compressing longer sequences into fixed-size representations.
- c. None of the other options is correct.
- d. It focuses on training models to explicitly output the reasoning steps required for solving a task, instead of directly giving the answer.
- e. It introduces the ability to train models with fewer data points by leveraging pre-trained embeddings.
- f. It allows models to perform tasks by conditioning on examples provided in the input, without requiring task-specific fine-tuning.
- g. It enables models to store task-specific knowledge directly in their architecture, removing the need for example inputs.

Domanda 3 (2 points (no penalty for a wrong answer))

Consider the following function:

```
1 def calculate_total_cost(items, discount, tax_rate, coupon_code,
2   membership_status):
3     total_cost = sum(items)
4     discounted_cost = total_cost * (1 - discount/100)
5
6     if coupon_code == "SUMMER21":
7         discounted_cost *= 0.8 # Apply an additional 20% discount
8
9     final_cost_with_tax = discounted_cost * (1 + tax_rate / 100)
10
11    if membership_status:
12        final_cost_with_tax *= 0.9 # Premium members get an additional 10% off
13
14    if final_cost_with_tax < 0:
15        final_cost_with_tax = 0
16
17    return final_cost_with_tax
```

An LLM engine generates the following test cases:

- **T1:** `assert calculate_total_cost([100, 50], 10, 5, "SUMMER21", True) == 102.06`
- **T2:** `assert calculate_total_cost([200, 100], 20, 10, "WINTER21", False) == 264`
- **T3:** `assert calculate_total_cost([50], 0, 0, "", False) == 50.0`
- **T4:** `assert calculate_total_cost([120, 80], 15, 5, "SUMMER21", False) == 162.8`

a) What is the pass rate of the test suite against the correct code?

b) Considering the following mutations:

- Line 3: `discounted_cost = total_cost * (1 - discount/10)`
- Line 6: `discounted_cost * (1 - tax_rate/100)` (Note: likely meant as assignment replacement or operator change)
- Line 9: `if final_cost_with_tax == 0:`

What is the mutation score of the test suite?

[Note: discuss and justify the results if needed]

Domanda 4 (1 point (-25% penalty for each wrong answer))

Adapter layers are introduced to reduce the number of tunable parameters. They consist in:

- a. A task-specific head added to the model, which is fine-tuned while the rest of the model is kept frozen.
- b. A method for splitting a large model into smaller sub-models that are trained independently and later combined.

- c. Replacing the feedforward layers in Transformers with low-rank matrix decompositions to reduce memory usage.
 - d. Small, trainable layers inserted into a pre-trained model, which are fine-tuned while the rest of the model is frozen.
 - e. A technique to merge multiple pre-trained models into a single model with fewer parameters.
 - f. Additional layers that replace the attention mechanism in the Transformer architecture to simplify computation.
 - g. None of the others is a valid description of adapters.
-

Domanda 5 (1 point (-25% penalty for each wrong answer))

Which of the following is/are not a technique to evaluate the quality of test cases generated with LLMs?

- a. Execution Success Rate
 - b. Coverage
 - c. Mutation score
 - d. Functional Correctness
 - e. Precision and recall
-

Domanda 6 (2 points (no penalty for a wrong answer))

For the following encoder-only transformer model, compute:

1. The number of parameters for the embedding layer (token + positional).
2. The number of parameters of the transformer (excluding the embedding layer or head layer(s)).

Note: assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

Model Specifications:

- Vocabulary size $V = 50,265$
- Embedding/model dimension $d = 1024$
- Positional embeddings: absolute, learned, for 514 positions
- Number of layers $L = 24$
- Number of attention heads $H = 16$
- FF-NN projection dimension: 4096
- Head dimension $d_k = d_v = 64$

Report the results in Millions (M) of parameters, rounded to the nearest integer.

Domanda 7 (2 points (no penalty for a wrong answer))

You are given the following sequence of characters:

aaaabcdddcdd

Initially, each character is a separate token. Apply Byte-Pair Encoding to produce 3 additional tokens. What are the generated tokens?

Write each token, in the correct order. Separate the merged tokens with a comma (e.g., if merged tokens are 'aa' and 'b', write: 'aa,b').

Domanda 8 (1 point (-25% penalty for each wrong answer))

How does an adversarial agent interaction scheme work?

- The agents work concurrently towards an individual goal, with the objective of maximizing their own benefit.
 - The agents work concurrently towards a common goal, with the objective of maximizing the overall benefit.
 - The agents work together towards a common goal, in an ordered sequence of subtasks.
 - The agents work together towards a common goal, in an unordered sequence of subtasks.
 - The agents work independently towards a common goal that is then compared.
-

Domanda 9 (1 point (-25% penalty for each wrong answer))

The main goal of instruction tuning is:

- To fine-tune language models to follow explicit instructions.
 - To reduce the size of the model without sacrificing performance.
 - To focus on improving the model's understanding of rare or unseen tokens.
 - None of the others is the main goal of instruction tuning.
 - To increase the model's tokenization speed by optimizing the vocabulary size.
 - To train language models to independently learn new languages from scratch.
 - To enhance the model's ability to generate longer sequences without truncation.
-

Domanda 10 (3 points (no penalty for a wrong answer))

The goal-oriented requirements engineering (GORE) is a technique where the requirements are constructed in three steps:

- user (or agent)
- high-level requirement

- low-level requirement

Consider the case where you receive natural language requirements in a string (called REQS). Describe an architecture of LLAMA agents to perform the steps of the GORE technique, and the prompt engineering techniques employed in each step.

For each agent, write the prompt including System, context, possible inputs from previous prompts, and expected output. *Notice: it is not necessary to write the python calls to the transformers or code for output parsing.*

Domanda 11 (2 points (no penalty for a wrong answer))

The attention for matrices Q, K, V (queries, keys, values) is computed as follows:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Where d_k is the dimensionality of the keys.

The following are the input tokens that reach an attention layer:

$$x = \begin{bmatrix} 3 & 4 & 1 \\ 0 & 4 & 0 \end{bmatrix}$$

The following are the W_q, W_k, W_v matrices for the attention layer:

$$W_q = \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix}, \quad W_k = \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix}, \quad W_v = \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Compute the output of the attention operation for the **first token**. Report at least 4 significant figures.

Domanda 12 (1 point (-25% penalty for each wrong answer))

A software repository-based dataset is:

- Any dataset that is based on data extracted from version control systems.
 - Any dataset that is composed by snippets of code.
 - Any dataset that is composed by fully working software code artefacts.
 - Any dataset that is based on open-source code available on online repositories.
 - Any dataset that is composed by production code artefacts paired with additional information.
-

Soluzioni

Domanda 1:

- It is possible to reduce the amount of false information provided to the final user.
- It is possible to prevent the obsolescence of answers by using up-to-date documents.

Domanda 2:

- (f) It allows models to perform tasks by conditioning on examples provided in the input, without requiring task-specific fine-tuning.

Domanda 3:

- **Pass rate:** All tests pass except for T4 where the result is 142.8 against the expected 162.8. Pass rate = 75.0%.
- **Mutation score:**
 - T1 is able to kill the first mutant.
 - T1 is able to kill also the second mutation (92.34 against expected 102.06).
 - None of the tests run line 9 so the third mutation is impossible to kill.
 - Mutants killed: 2/3 → 66.7%.

Domanda 4:

- (d) Small, trainable layers inserted into a pre-trained model, which are fine-tuned while the rest of the model is frozen.

Domanda 5:

- (d) Functional Correctness

Domanda 6:

- **Embedding size:** 52 M
- **Transformer size:** 302 M
- *Details:* The described model is RoBERTa Large.
 - Embeddings: $50265 \times 1024 + 514 \times 1024 \approx 52 \text{ M}$.
 - Transformer: $24 \times (4 \times 1024 \times 1024 + 2 \times 1024 \times 4096) \approx 302 \text{ M}$.

Domanda 7:

- 1st merge: d , d
- 2nd merge: a , a
- 3rd merge: c , dd

Domanda 8:

- (a) The agents work concurrently towards an individual goal, with the objective of maximizing their own benefit.

Domanda 9:

- (a) To fine-tune language models to follow explicit instructions.

Domanda 10: Architecture description: To extract the three points "user, high and low level requirement" we can use a router chain to route the decision of an agent to three different subchains, each one specialized in one task. One subchain for the user extraction, one for the high level reqs and one for the low level ones.

Prompt engineering techniques adopted: Possible techniques adopted could be few-shot prompting.

Prompts:

- **User Extraction Agent:**

You are an expert in extracting user from natural language description of requirements.

##Example1: **natural language description...** ##Output: Customer, Library Owner...

Now please, with the following information extract the relevant user. {REQS}

- **High-Level Requirement Agent:**

You are an expert in extracting high-level requirements from natural language description of requirements.

##Example1: ... ##Output: Customer could buy book...

Now please, with the following information extract the relevant high-level requirements. {REQS}

- **Low-Level Requirement Agent:**

You are an expert in extracting low-level from natural language description of requirements.

##Example1: ... ##Output: Codebase should implement a 'Book' class...

Now please, with the following information extract the low-level requirements. {REQS}

Domanda 11:

- Dimension 1: 3.1956
- Dimension 2: 1.6089
- Dimension 3: -5.6310

Domanda 12:

- (a) Any dataset that is based on data extracted from version control systems.